L45 — Recap: Designing Efficient Data Systems

Unit: 3 | **Chapter:** Synthesis | **BT Level:** BT6 | **CO:** CO4, CO5

Learning Objectives

- Synthesize the entire ADSA course into a unified decision framework
- Select appropriate data structures and algorithms for real-world problems
- Analyze trade-offs between time, space, and implementation complexity
- Design a complete data system given requirements

---

1. The Data Structure Selection Framework

Choosing the right data structure requires understanding the **access patterns** and **operation frequencies** of your system.

```
Ask these questions:
1. What is the primary operation? (search, insert, delete, sort, traver
2. Are keys ordered or unordered?
3. Is data size fixed or dynamic?
4. What are the time and space constraints?
5. Is worst-case or average-case performance critical?
```

---

2. Complete Data Structure Decision Guide

2.1 By Primary Operation

Primary Need	Best Data Structure	Time
LIFO access	Stack	O(1) push/pop
FIFO access	Queue	O(1) enqueue/dequeue
Min/Max extraction	Heap (Priority Queue)	O(log n)
Ordered key lookup	BST / AVL / Red-Black	O(log n)
Unordered key lookup	Hash Table	O(1) average
Range queries	B-Tree / Sorted Array	O(log n + k)
Sorted output	Any BST (inorder) / Sorted Array	O(n)
Prefix matching	Trie	O(key length)

2.2 By Data Characteristics

Characteristic	Recommended Structure
Fixed size, random access	Array

Dynamic size, sequential	Linked List
Hierarchical (parent-child)	Tree
Peer connections	Graph
Small, frequently sorted	Insertion Sort
Large, general purpose	Merge Sort or Timsort
Distinct keys, fast lookup	Hash Table

---

3. Algorithm Complexity Quick Reference

Sorting Algorithms

Algorithm	Best	Average	Worst	Space	Stable
Insertion Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Bubble Sort	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No
Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes
Quick Sort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1)	No

Graph Algorithms

Algorithm	Time	Space	Notes
BFS	O(V + E)	O(V)	Shortest path (unweighted)
DFS	O(V + E)	O(V)	Cycle detection, toposort
Dijkstra	O((V + E) log V)	O(V)	Shortest path (non-neg weights)
Bellman-Ford	O(VE)	O(V)	Negative edges, cycle detection
Warshall	$O(V^3)$	$O(V^2)$	All-pairs reachability

Data Structure Operations

Structure	Search	Insert	Delete	Space
Array (sorted)	O(log n)	O(n)	O(n)	O(n)
Linked List	O(n)	O(1)	O(1)*	O(n)
Stack	—	O(1)	O(1)	O(n)
Queue	—	O(1)	O(1)	O(n)
Binary Heap	O(n)	O(log n)	O(log n)	O(n)
BST (balanced)	O(log n)	O(log n)	O(log n)	O(n)
AVL Tree	O(log n)	O(log n)	O(log n)	O(n)
B-Tree (order t)	O(t log_t n)	O(t log_t n)	O(t log_t n)	O(n)
Hash Table	O(1) avg	O(1) avg	O(1) avg	O(n)

---

4. Design Case Studies

Case Study 1: Design a Contact Book App

Requirements: Store N contacts, search by name, add/remove contacts.

Analysis:

- Search by name → hash table (O(1) average) or sorted list (O(log n) binary search)
- Alphabetical listing → sorted array or BST inorder
- Frequent updates → dynamic structure (not sorted array)

Decision: Hash table (name → contact) for lookups + sorted array for display.

```
class ContactBook:
    def __init__(self):
        self.contacts = {}      # Hash table: name → details
        self._sorted = []       # Sorted list for display

    def add(self, name, phone):
        self.contacts[name] = phone
        if name not in self._sorted:
            import bisect
            bisect.insort(self._sorted, name)

    def search(self, name):
        return self.contacts.get(name)   # O(1)

    def list_all(self):
        return [(n, self.contacts[n]) for n in self._sorted]  # Sorted

    def delete(self, name):
        if name in self.contacts:
            del self.contacts[name]
            self._sorted.remove(name)
```

Case Study 2: Task Scheduler (OS Ready Queue)

Requirements: Add tasks with priorities; always execute the highest-priority task next.

Analysis: Need fast extract-min → **min-heap (priority queue)**.

```
import heapq

class TaskScheduler:
    def __init__(self):
        self.heap = []
        self.counter = 0   # Break ties by arrival order

    def add_task(self, priority, name):
        heapq.heappush(self.heap, (priority, self.counter, name))
        self.counter += 1

    def next_task(self):
```

```
        if not self.heap:
            return None
        priority, _, name = heapq.heappop(self.heap)
        return name, priority
```

Case Study 3: Route Planner (GPS)

Requirements: Given a road network, find shortest path between any two points.

Analysis:

- Graph with V cities, E roads
- Non-negative distances → **Dijkstra's algorithm**
- Pre-computed all-pairs → **Warshall / Floyd-Warshall**

Implementation pattern:

```
# Build adjacency list from road network
graph = {}
for road in roads:
    city_a, city_b, distance = road
    graph.setdefault(city_a, []).append((city_b, distance))
    graph.setdefault(city_b, []).append((city_a, distance))  # Bidirect

# Query
path, length = dijkstra_with_path(graph, source='A', target='Z')
```

Case Study 4: Search Engine Index

Requirements: Index millions of documents; support fast keyword search.

Analysis:

- Map words → document IDs: **Inverted Index** (hash table: term → posting list)
- Posting list sorted by document ID for Boolean intersection
- Boolean AND = merge two sorted posting lists in O(m + n)

```
def boolean_and_search(index, term1, term2):
    """Merge sorted posting lists — O(m + n)"""
    list1 = index.get(term1, [])
    list2 = index.get(term2, [])
    i, j = 0, 0
    result = []
    while i < len(list1) and j < len(list2):
        if list1[i] == list2[j]:
            result.append(list1[i])
            i += 1; j += 1
        elif list1[i] < list2[j]:
            i += 1
        else:
            j += 1
```

```
    return result
```

---

5. Key Principles for Efficient Systems

1. **Choose data structures to match access patterns** — hash for equality lookup, BST/sorted array for ranges, heap for priority access.
2. **Profile before optimizing** — measure actual bottlenecks; premature optimization wastes time.
3. **Amortized analysis matters** — a structure with occasional O(n) operations but O(1) amortized (like dynamic array) may be better than O(log n) per operation.
4. **Consider constants** — O(n log n) with small constant beats O(n) with large constant for practical n.
5. **Cache performance** — array-based structures (heap, sorted array) outperform pointer-based ones (linked list, tree) for large n due to spatial locality.
6. **Trade-off space for time** — hash tables, memoization, prefix sums sacrifice memory to gain speed.

---

6. Course Summary Map

```
ADSA
├── Unit 1: Fundamentals
│   ├── Data, Information, ADT
│   ├── Complexity: O, Ω, Θ
│   ├── Arrays: 1D, 2D, Sparse
│   ├── Sorting: Bubble, Insertion, Selection, Quick, Merge
│   └── Linked Lists: SLL, DLL, Circular
│
├── Unit 2: Linear Structures
│   ├── Stacks: PUSH/POP, postfix eval, infix→postfix
│   ├── Recursion: base case, call stack, classic problems
│   └── Queues: Queue, Deque, Priority Queue
│
└── Unit 3: Advanced Structures
    ├── Graphs: BFS, DFS, Dijkstra, Bellman-Ford
    ├── Trees: Binary, BST, AVL, B-Tree
    ├── Heaps: Min/Max, Heap Sort
    ├── Hashing: Hash functions, Chaining, Open Addressing
    └── File Organization: Sequential, Relative, ISAM, Inverted
```

Summary

- Data structure selection depends on: **primary operation, access pattern, size, ordering requirements**.

- No single "best" structure — every choice is a trade-off.
- **Practical rule:** Use a hash table for unordered lookups, BST/sorted structure for ordered access, heap for priority, graph algorithms for connectivity problems.
- Always verify correctness first, then optimize performance for measured bottlenecks.

References

- Cormen et al., *Introduction to Algorithms* (MIT Press, 2022) — comprehensive reference
- Skiena, *The Algorithm Design Manual* (Springer, 2020) — practical applications
- Laaksonen, *Guide to Competitive Programming* (Springer, 2017) — competitive use cases